Java Programming ++

Dipa Joshi

1. 5x8 = 40 2079

1. What is event handling? Describe the component of event handling by showing example of mouse event.

2. Explain the process of creating menu bar with eg

3. Explain RMI structure in brief. Diff bet^{n} RMI and CORBA.

4. What is JDBC driver. What are the steps to connect Java application to database?

5. What are the advantage of JSP? Write a JSP program to find cube a number.

6. Why is layout manager important? Explain Grid layout with eg.

7. Define result set? How do you establish database connectivity? 2078

8. Define layout manager. Explain Border layout with eg.

9. Define CORBA. Write down CORBA service in brief.

10. What is Java Servlet? Describe HTTP request-response model of servlet

11. Define JSP. Write down advantage of JSP. Explain JSP scripting elements. 2078

2076

12. Explain Two ways of handling events briefly.

13. Diff b/w Java swing and AWT. Create a simple JFrame with title.

14. What is RMI? Explain RMI structure in brief.

15. What is JDBC? Write down steps to connect to db in java

16. What is Servlet? Discuss life cycle of servlet in brief.

17. Explain Internal frame with eg.

18. Describe features of Java in brief.

19. short note 3 1500

1. JSP
2. CORBA
3. MENV BAR 2076

20. What is socket? How can you communicate two programs in a network using TCP socket?

21. What are the features of swing

22. Discuss the process of event handling in Java

23. What is JDBC? Explain various type of JDBC drivers, also explain major desin constraints with JDBC

4/12/27 13:08

Long Answer Question 3x12 = 36

1. What is Java socket? Dicsuss different methods of creating connection and connectionless sockets in java with the help of suitable program.

2. Define Servlet. Discuss lifecycle of servlet. Diff bet^{n} servlet and JSP.

3. Compare swing and AWT. Write GUI Program to find sum of 2 no's using swing components. Create Textfields for inputting 2 nos and a button named SUM. Program should produce sum when use clicked on SUM button.

4. Short note
   a) MVC design pattern
   b) Prepared statement in JDBC.

5. Diff AWT and swing. Describe swing container hierarchy brief

6. What is socket? What are the steps for creating client side and server side of application using TCP.

7. Explain RMI architecture with diagram.

8. Define layout manager. Explain Grid layout and border layout manager with eg of your choice.

9. Define socket. Write down steps to create client and server program implement the client server chat program in java.

1 What is event handling? Describe the component of event handling by showing example of mouse event

Ans° Event handling is the mechanism that controls the event and decides what should happen if an event occurs. Java uses the Delegation Event model to handle the events.

→ The main components of Event handling are

i) Event: An object that describes a change in state, such as a button click or keystroke

ii) Event Source: Events are generated by source such as bottons, checkboxes, list, scrollbar etc.

iii) Listners: Listeners are used for handling the events generated from the source.

Example:

```
import Java.awt.*;
import java.awt.event.*;
import Javax.swing.*;
public class MouseListnerExample extends JFrome
  implements MouseListner{
 private JButton button;
  public MouseListnerExample(){
   setTitle("Handling Mouse Listner");
   setSize(300, 150);
   button.addMouseListner(this);
   setVisible(true);
  }
public void mouseEntered(MouseEvent e){
 button.setBackground(Color.RED);
 button.setText("Clicked!");
}
```

```
public void mouseEntered(MouseEvent e) {
    button.setBackground(color.RED);
}
public void mouseExited(MouseEvent e) {}
public void mousePressed(MouseEvent e) {}
public void mouseReleased(MouseEvent e) {}
public static void main(String[] args) {
    new MouseListnerExample();
}
}
```

2. Explain the process of creating Menu bar with eg.

Ans° Swing provides support for pull-down and popup menus. A JMenuBar can contains several JMenus. Each of the JMenus can contain a series of JMenuItem that you (we) can select

The process (steps) of creating menu in Java are

i) Create a frame using JFrame.
ii) Create a menu bar using JMenuBar.
iii) Create a menu using JMenu.
iv) Create a menu item using JMenuItem.
v) Add menu items to the menu using menu.add(itemname)
vi) Set the menu to menubar using mb.add(menu)
vii) Set the menubar to the frame using d.setMenuBar(mb)

Example:

Example

```
import Javax.swing.*;

class MenuExample
{
   JMenu menu;
   JMenuItem i1; i2; i3;
   MenuExample()
    {
JFrame f = new JFrame ("Menu Example");
JMenuBar mb = new JMenuBar();
        menu = new JMenu ("menu");

i1 = new JMenuItem ("item 1");
i2 = new JMenuItem ("item 2");
i3 = new JMenuItem ("item 3");

        menu.add (i1);
        menu.add (i2);
        menu.add (i3);

        mb.add (menu);

      f.setMenuBar (mb);
      f.setSize(400, 400);
      f.setVisible (true);
    }
Public static void main (string[] args)
   {
      new MenuExample();
   }
}
```

3. Explain RMI structure in brief. Diff bet^{w} RMI cor

Ans° RMI stands for Remote method Invocation. It is a mechanism that allows an object residing in one system (JVM) to access or invoke an object running on another JVM.

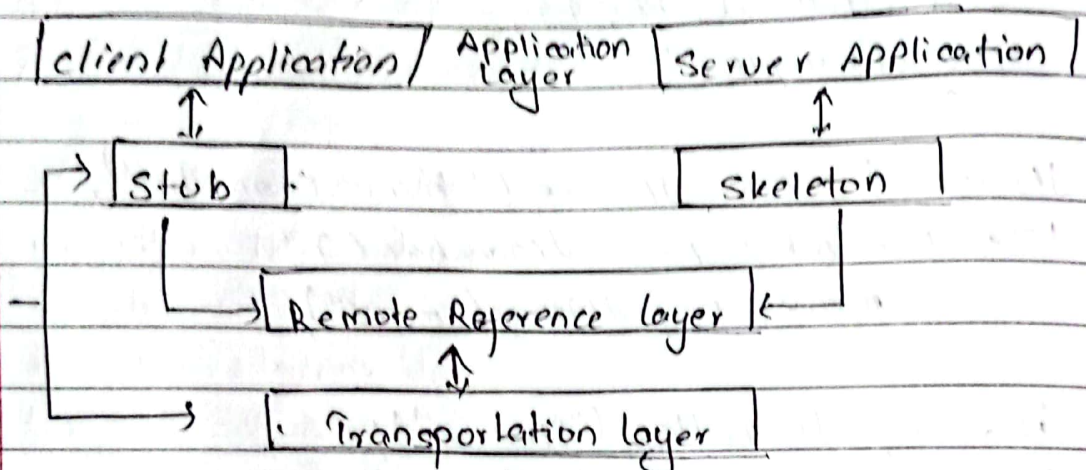


Fig: RMI architecture

1. Application layer

This layer is the actual system, i.e. client and server, which are involved in communication. The java program on client-side communicates with the java programs on server-side.

2. Proxy layer or stub | skeleton layer

The proxy layer is responsible for managing the remote object interface bet^{w} client and server. In this layer, the stub class is the client-side proxy for the remote object and skeleton class is the server-side proxy for the remote object.

The skeleton class is another proxy lives with the real object on its original host and receives RMI (remote method invocation) from stub and pass them to the object.

3. Remote Reference Layer (RRL)

The RRL layer defines and supports invocation semantics of the RMI connection. This layer maintains session during the method call. The RRL layer provides a RemoteRef object that represents the link to the remote service implementation object.

4. Transportation Layer

The transportation layer is responsible for setting up connection and transportation of data from one machine to another.

The default connection is set up only in TCP/IP protocol.

	RMI	CORBA
1.	It stands for remote method invocation.	It stands for Common Object Request Broker Architecture
2.	RMI is a java specific technology.	CORBA is independent of programming languages
3.	It uses java interface for implementation.	It uses Interface Definition Language (IDL) to seperate interface from implementation
4	It passes objects by values	It passes objects by reference.
5.	RMI is slow in execution	Fast in execut^n than RMI
6.	Java RMI is a server-centric model	CORBA is a peer-to-peer system

Q. What is JDBC driver. What are the steps to connect Java application to the database

Ans: A JDBC driver is a software component enabling a java application to interact with a database JDBC drivers are client-side adapters that convert requests from java programs to a protocol that the DBMS can understand.

Steps to connect to the database

i) Register the driver class
ii) Create Connection
iii) Create statement
iv) Execute Queries
v) Close Connection

i) Register the driver class
The forName() method of class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax for forName() method :-

```
public static void forName(String className) throws
        ClassNotFoundException.
```

ii) Create the connection object :.
The getConnection() method of DriverManager class is used to establish Connection with the database.

Syntax of getConnection() method :.

```
public static void Connection getConnection (string url) throws
        SQL Exception
```

iii) Create the statement object :-

The createStatement() method of connection interface is used to create statement. The object of statement is responsible to create queries with database.

Syntax of createStatement() method :-

public statement createStatement() throws SQLException

iv) Execute the query :-

The executeQuery() method of statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of table.

Syntax of executeQuery() :-

public ResultSet executeQuery (String url) throws SQLException.

v) Close the connection object :-

By closing connection object statement and ResultSet will be closed automatically.

The close() method of connection interface is used to close the connection.

Syntax of close() method :-

public void close() throws SQLException.

5. What are the advantage of JSP ? Write a JSP program to find cube number.

Ans°.
- User need not write HTML and JAVA code seperately
- JSP can be used for both front end and for writing business logic.
- JSP is dynamic compilation, which means when a JSP is modified, it need not be compiled and restarted in the web server. After the modification of JSP, refresh the browser, changes will be reflected.
- JSP is Efficient : Every request for a JSP is handled by a simple Java thread.
- In a JSP pages visual content and logic are seperated, which is not possible in Servlet.

Why is layout manager important? Explain Grid layout with eg. Note: Also write all note of layout

° Introduction :-

A layout manager is an object that controls the size and position (layout) of components inside a container object. eg. a window is a container that ~~control~~ contains components such as buttons and labels.

⇒ Layout managers are important because they control the arrangement of components in a container, which is essential for designing user interfaces that are responsive and adjustable to diff screen sizes.

⇒ Types

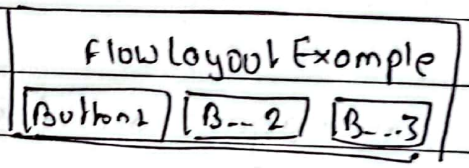
1. FlowLayout

FlowLayout is a simple layout manager that arranges components in a row, left to right, wrapping to the next line as needed. It is ideal for scenarios where components need to maintain their natural sizes and maintain a flow-like structure.

Example:

```
import javax.swing.*;
import java.awt*;
public class FlowLayoutExample {
    public static void main(String[] args) {
    {
        JFrame frame = new JFrame("FlowLayout Example");
        frame.setLayout(new FlowLayout());
        frame.add(new JButton("Button 1"));
        frame.add(new JButton("Button 2"));
        frame.add(new JButton("Button 3"));
        frame.pack();
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOS
    }
}
```

Output



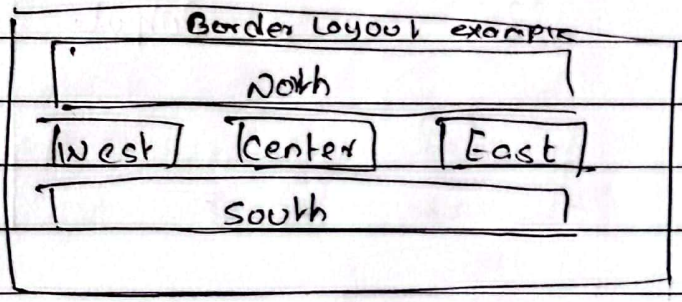
2. BorderLayot

Borderlayout divides the container into five regions: North, South, EAST, WEST and CORNER. Components can be added to these regions, and they will occupy the available space accordingly. This layout manager is suitable for creating interfaces with distinct sections, such as a titlebar, content area and status bar.

Example :-

```
import java.swing.*;
import java.awt.*;
public class BorderLayoutExample
{
  public static void main(String[] args)
  {
   JFrame frame = new JFrame("BorderLayout Example");
   frame.setLayout(new BorderLayout());
   frame.add(new JButton("North"), BorderLayout.NORTH);
   frame.add(new JButton("South"), BorderLayout.SOUTH);
   frame.add(new JButton("East"), BorderLayout.EAST);
   frame.add(new JButton("West"), BorderLayout.WEST);
   frame.add(new JButton("Center"), BorderLayout.CENTER);
   frame.pack();
   frame.setVisible(true);
   frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }
}
```

Output



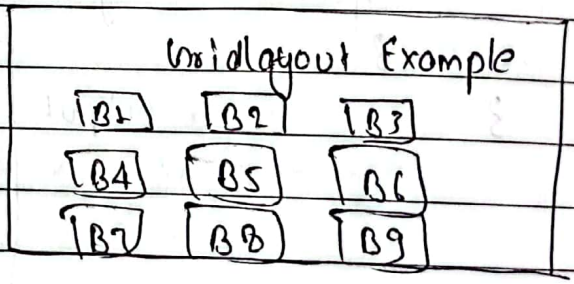
3. Grid layout

GridLayout arranges components in a grid with a specified number of rows and columns. Each cell in the grid can hold a component. This layout manager is ideal for creating a uniform grid of components, such as a calculator or a game board.

Example :-

```
import java.swing.*;
import java.awt.*;
public class GridLayoutExample
{
  public static void main(String[] args)
  {
    JFrame frame = new JFrame("GridLayout Example");
    frame.setLayout(new GridLayout(3,3));
    for(int i=1; i<=9; i++)
    {
      frame.add(new JButton("Button"+i));
    }
    frame.pack();
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
  }
}
```

Output.



4. CardLayout

CardLayout allows components to be stacked on top of each other, like a desk of cards. Only one component is visible at a time, and you can switch bet^{w} components using methods like next() and previous(). This layout is useful for creating wizards or multi-step processes.

Example:-

and so on of Javapoint web.

Google

7. Define result set ? How do you establish database connectivity.

Ans° ResultSet is a table of data where each row represents a record and each column represents a field in the database. The resultSet has a cursor that points to the current row in the ResultSet and we can navigate in ResultSets by using next(), first() and last() and previous() methods.

↑ ans before for 2nd part.

8. Define CORBA. Write down CORBA service in brief.

Ans° The Common object Request Broker Architecture CORBA is a standard defined by the Object managment group OMG that enables software components written in multiple computer languages and running on multiple computers to work together.

CORBA Services

→ Object life cycle : Defines how CORBA objets are created, removed, moved and copied.

→ Naming : Defines how CORBA objects can have friendly symbolic names

→ Events : Decouples the communication bet^n distributed objects.

→ Relationships : Provides arbitrary typed n-ary relationships between

→ CORBA objects Externalization : Coordinates the transformation of CORBA objects to and from external media

Transactions : Coordinates atomatic access to CORBA objects.
Concurrency Control : Provides a locking service for CORBA objects in order to ensure serializable access.
Property : Supports the association of name-value pairs with CORBA objects.
Trader : Supports the finding of CORBA objects based on properties describing the service offered by an object
Query : Supports queries on objects.

9.

9. What is JavaServlet ? Describe HTTP request-response model of Servlet.

Ans° Java Servlets are the java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the web server, process the request, produce the response, and then send a response back to the web server.

→ The HTTP Request-Response Model in a servlet involves the exchange of data bet^w a client and a server using the HTTP protocol. This model is central to web applications and works as follows :

1. Client Sends an HTTP Request : The client sends a HTTP request to the server. The request consists of Request line (URL), Headers (content type) and body (data).

2. Server Recieves the request

The web server forwards the request to the servlet container.

3. Servlet processes the Request

The servlet's service() method is called by the container. The service() method delegates the request to specific methods like doGet() or doPost() for handling requests.

- The servlet can read request data using HttpServletRequest object.
- Perform logic such as database queries or validation and generates dynamic content.

4. Server Sends an HTTP Response

The Servlet generates a response using the HttpServletResponse object.

The response consists of status code, Headers, body.

The Container sends the response back to the client.

10. Define JSP ? Write down advantages of JSP. Explain JSP scripting elements

Ans° JavaServer Pages is a technology used to create dynamic, server-side web pages. It allows developers to embed Java code directly into HTML using special JSP tags, making it easier to generate dymomic content based on user requests.

→ Advantages of JSP

i) Seperation of Concerns : keeps HTML and Java logic seperate for easier maintenance.

ii) Ease of Development : Simplifies dynamic web page creation by embadding Java code directly in HTML.

iii) Platform Independence : Works on any platform that supports Java.

iv) Built-in Tag Libraries : Reduces Java code with libraries like JSTL

v) Java Integration : Easily integrates with other Java technologies. (eg. JDBC, EJB).

vi) Error handling : Provides custom error pages and robust exception handling.

vii) Reusability : Allows reuse of components like JavaBeans and custom tags.

→ The scripting elements provides the ability to insert java code inside the JSP. There are 3 types of scripting elements :

- Scriptlet tag
- Expression tag
- Declation tag

JSP scriptlet tag

A scriptlet tag is used to execute java source code in JSP.

Sytax: <% java source code %>

Example

```
<html>
<body>
<% out.print("wlcm to jsp"); %>
</body>
</html>
```

JSP expression tag

Embeds Java expressions inside HTML that get evaluated and the result is directly sent to client. An expression evaluates a java expression and automatically sends the result to output stream. This makes it simpler than scriptlet cause you do not need out.println() method to send output

Syntax: <%= expression %>

Example: <% = "Hello," + userName + "!" %>

or <% = "wlcm to JSP" %>

Declaration tag

Used to declare variables and methods that are used throughout the JSP page.

Syntax: <%! dataType variableName; %>

<%! returnType methodName()

{ // method body } %>

Eg.

```
<%! int count = 0; %>
<%! public void
incrementCount() { count++; } %>
```

11. Explain two ways of handling events briefly.

Ans° Java provides two primary ways to handle events in GUI applications.

1. Delegation Event Model (standard Approach)

The Delegation Event Model is the modern, widely used way of handling events in Java. It seperat the event source from the event listner.

key components:

- Event Source: The component on which the event occurs (eg. Button, Textfield).
- Event Listner: An object that implements a listner interface (e.g. ActionListner, MouseListner).
- Event Registration: The listner is registered with the event source using methods like AddActionLisher() or addMouseListener().

Example:

```
button.addActionListner (e ->
    System.out.println ("Button clicked!");
```

Advantages:

- promotes mudularity by seperating event-handling logic.
- Supports reusable and scalable code.

2. Using Anonymous Inner Classes

This approach is a part of Delegation Event Model but uses anonymous inner classes for event-handling logic. It is concise and suitable for small programs where defining a seperate listner class is unnecessa

Explanation:
The listner interface is implemented inline, directly within the event registration code.
It avoids the need to create a seperate listner class.
Example:

```
button.AddActionListner(new ActionListner() {
{
public void actionPerformed(ActionEvent e)
{
System.out.println("Button clicked!");
}
});
```

Advantages
Reduces boilerplate code.
Simplifies event handling for single-use ~~eta~~ cases.

Difference betw Java swing and AWT. Create a simple JFrame with title.

Java AWT	Java swing
AWT componets are platform-dependent.	Swing components are platform-independent
AWT components are heavyweight.	Are lightweight.
i) Doesn't support pluggable look & feel.	It supports pluggable look & feel
ii) Provide fewer components than swing	Provide more powerful components
iii) AWT doesn't follows MVC	swing follows MVC

Example:

```
import javax.swing.JFrame;
public class SimpleJFrame
{
    public static void main (String [] args)
    {
        // create a JFrame instance
        JFrame frame = new JFrame("My first JFrame");
        // set the default close operation
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // set the size of JFrame
            frame.setSize(400, 300);
    // Make the JFrame visible
            frame.setVisible(true);
        }
}
```

13. What is RMI? Explain RMI Structure in brief.

Ans°

RMI stands for Remote Method Invocation. It is an API that provides a mechanism to create distributed application in java. It is a mechanism that allows an object residing in one system (JVM) to access or invoke an object running on another JVM.

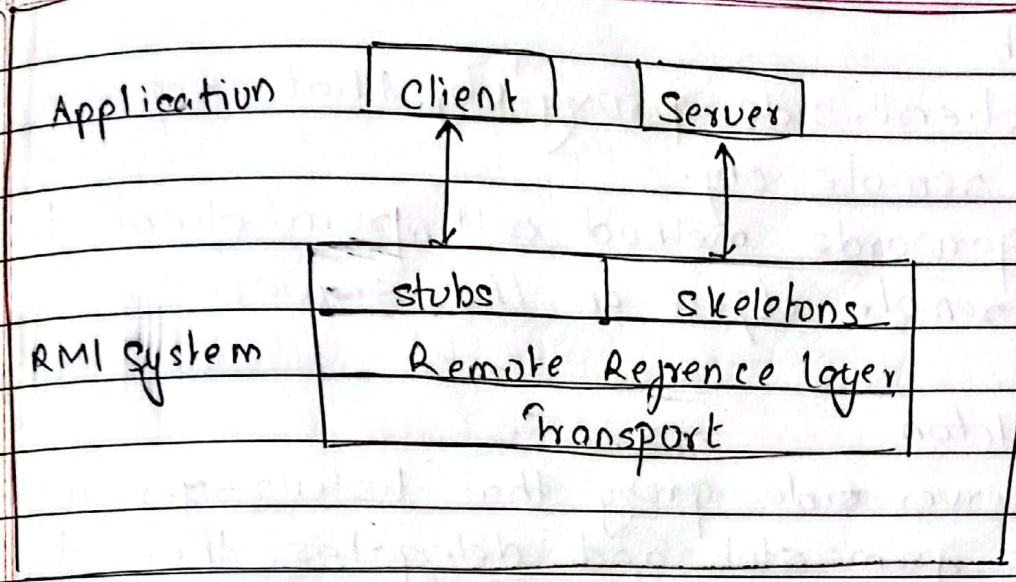


Fig. RMI architecture

1. Client
   - The client is the application that invokes a method on a remote object.
   - It uses a stub (a proxy obj) to communicate with the remote object.

2. Server
   - The Server hosts the remote obj and provides its implementation.
   - It registers the remote object with RMI Registry so that clients can locate it.

3. RMI Registry
   - A lookup service where remote objs are registered.
   - Clients use registry to find and obtain references to remote objects.

4. Stub
- A client-side proxy obj that represents the remote obj.
- It forwards method call from client to the remote object on the server.

5. Skeleton
- A server-side proxy that listens for method calls from stub and delegates them to the actual remote object.

6. Remote Reference Layer
- Handles communication details b/w client & server. It
- Manages the flow of method calls b/w client and server including serialization of args and responses.

7 Transport Layer
- Manages low-level communication b/w JVMs, typically over TCP/IP.

14. What is Servelet? Discuss it lifecycle.

Ans A Servlet is a java programming language class that is used to extend the capacities of servers that host application accessed by means of a request-response programming model

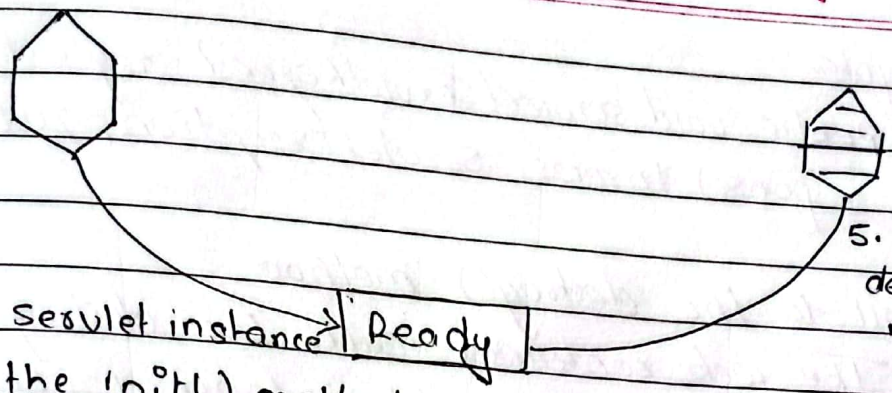


Fig: Servlet life cycle

1. Loading Servlet class

A servlet class is loaded when first request for the servlet is received by web container.

2. Servlet instance Creation

After the servlet class is loaded, web container creates the instance of it. Servlet instance is created only once in life cycle

3. call the init() method

init() method is called by web container on servlet instance to initialize the servlet.

Syntax:

```
public void init (ServletConfig config) throws
                servlet_Exception
```

4. Call to the service() method

The web container calls the service() method each time when the request for servlet is recieved. The service() method will then call doGet() or doPost() method based on type of HTTP request

Syntax:

public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException.

5. Call to the destroy() method

The web container calls the destroy() method before removing servlet instance from the service. It gives the servlet an opportunity to cleanup any resource for eg. memory, thread etc.

Syntax:

public void destroy()

15 Explain internal frame with example.

Ans° An internal frame in java refers to a lightweight, movable, and resizable window that exists within a main application window (typically a JFrame). It is implemented using the JInternalFrame class, which is a part of the Swing library. Internal frames are commonly used in Multiple Document Interface (MDI) applications, where multiple documents or windows are managed within a single parent frame.

Features of JInternalFrame

- Can be minimized, maximized, or closed independently of the main frame.
- Exists only within the bounds of the parent container (usually a JDesktopPane).

- Supports adding components like buttons, text fields, and panels.
- Provides a title bar, border, and control buttons (minimize, max², close).

Example:

```
import Java.swing.*;
public class InternalFrameExample
{
  public static void main(string[] args)
   {
     JFrame jframe= new JFrame("Internal frame eg");
     jframe. setSize(500, 400);
     jframe. setDefaultCloseOperation (JFrame. EXIT_ON.(LOSE);

     JDesktopPane desktopPane = new JDesktopPane();
// Create an internal jframe
     JInternalFrame internalframe = new JInternalFrame
     ("Internal frame", true, true, true, true);
     internalframe. setSize(200, 150);
     internalframe. setVisible (true);

// Add the interal jframe to desktop Pane
     desktopPane. add (intrnalframe);

// Add the desktop pane to main jframe
     jframe. add (desktopPane);
     jframe. setVisible (true);
   }
}
```

16. Describe the features of Java in brief

Ans° The features of Java includes:

1. JVM (Java Virtual Machine)
Provides a runtime experiment for executing Java applications, ensuring platform independence

2. Automatic Memory Management (Garbage Collection)
Java automatically manages memory allocation & deallocation, reducing developer overhead.

3. Rich Standard Library
Java includes a vast library (Java API) for a data structures, networking, file IIO, database connectivity and more.

4. Write Once, Run Anywhere (WORA)
Java programs compiled into bytecode can run on any JVM, regardless of underlying hardware or OS.

5. Dynamic and Extensible
Java supports dynamic loading of classes & extensibity through its libraries and APIs.

6. Built-in Networking
Java simplifies network programming with its extensive networking capabilities and APIs.

7. High Performance
Java acheives high performance through Just-In-Time (JIT) compilation.

17. What is socket? How can you communicate two programs in network using TCP socket?

Ans: A socket is one end point of two way communication link betw two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent.

→ Sockets provide the communication mechanism betw two programs using TCP.

1. Creating client side of the application.
   i) Connect to a server (creation of socket obj).
   ii) Read from and write to a socket.
   iii) Close the connection.

```
Socket clientSocket = new Socket("hostname", "portnumber");
```

2. Creating server side of application
   - Create a server
   - Listen for the client request
   - Read input from & send output to client.
   - Close the connection.

OR

Define socket and client-server application using TCP.

18. What are the features of swing.

Ans° The features of swing are

1. Platform Independence
Swing is written in pure Java, making it platform-independent. Applications built using swing can run on any platform with a compatible JVM

2. Lightweight components
Swing components are lightweight, i.e they don't rely on native OS GUI components. Instead, they are drawn using Java's rendering capabilities.

3. Rich set of components
Swing provides a comprehensive set of GUI components like buttons, text fields, tables etc.

4. Pluggable look-and-feel
Swing allows applications to have customizable "Look-and-feel", enabling developers to change appearance and behaviour of components at runtime.

5. Event-Driven programming
Uses event delegation model to handle user interaction like button clicks and keyboard inputs.

6 Advanced Graphics support
Enables creation of visually rich interfaces.

7. Internationalization: Supports applications for multiple languages and regions.

Q. Discuss the process of event handling in Java.

Process of Event handling includes :-

* Event Source : The component that generates event i.e button, checkbox, text field etc.
* Event object :x

Event is generated using event source (button, CB, TF)
An object is created when an event occurs, encapsulating details about event eg. type, source.
Event Listener defines methods for handling specific events. which includes ActionListner, mouseListner and keyListner.

The listner is registed with event source using methods like AddActionListner(), addMouseListner() etc.

Listener method is implemented to handle the events.

Event Delegation: Event handling is delegated to the registered listner.

Event Handling Execution: When an event occurs, the registed Listner is notified, and the corresponding method (e.g. actionPerformed) is executed.

25. Define JDBC. Explain Types of JDBC also explain with major constraints with JDBC.

Ans° Java database Connectivity is an API (Applicat° progm. Interface) in Java that allows Java appl°s to connect to, interact with, and manipulate dbs, sending SQL queries and handling results.

Types:

1. Type-1 (JDBC-ODBC Bridge Driver):
This driver acts as a bridge bet° JDBC and open db connectivity. It translates JDBC API calls into ODBC calls using ODBC driver.

2. Type-2 (Native-API Driver):
This driver converts JDBC calls into db-specific native API calls. It relies on native libraries provided by the db vendor, making it faster but platform-dependent.

3. Type-3 (Network Protocol Driver):
This driver uses a middleware server to comm^{t} with db. JDBC calls are sent to middleware, which translate them into db-specific calls. It supports multiple dbs trough single driver

4. Type-4 (Thin Driver)
This is pure Java driver that directly converts JDBC calls into dbs native protocol. It doesn't require additional soft^{t} or middleware and is platform-independent

→ Major to design confains with JDBC are

i) Performance: Connect^{n} setup is resource-intensive
ii) Portability: Drivers are database-specific
iii) Error Handling: Complex exception management.
iv) Scalability: Struggles with high-traffic applications.
v) Limited Database features
vi) Driver Dependency.

Long question 12x1.

1. What is JavaSockets? Discuss diff method of creating connection and connectionless sockets in Java with the help of eg.

Ans° Java Sockets are endpoint for communication b/w two machines or processes. Java provides the Java.net package for socked programming enabling comm^t be over a net^w using TCP or Protocols.

1. Connection-Orinted Sockets (TCP)
TCP sockets establish a reliable connection b/w a client and a server, ensuring ordered and error-checked comm^t.

key Methods
i) ServerSocket
→ ServerSocket (int port): Creates a server listen a specified port.
→ accept(): waits for and accepts a client conn
ii) Socket
→ Socket (string host, int port): connects to a spe server and port
→ getInputStream() / getOutputStream(): Real

Example: Server:

```
ServerSocket server = new
ServerSocket (5000);
Socket client = server.accept(); // Accept client c
BufferedReader (new InputStreamReader (client.ge
    stream()));
System.out.println("Client: " + reader.readLine(
client.close();
server.close();
```

client:

```
socket socket = new Socket ("localhost", 5000);
PrintWriter writer = new PrintWriter(socket.getOutputStream
  (), true);
writer.println ("Hello, server!");
socket.close()
```

2. Connectionless Sockets (UDP)

UDP sockets allow communication without establishing a connection, providing faster but unreliable comm.

Key Methods

i) DatagramSocket

It creates a socket to send/ receive packets.

ii) Datagram Packet

- It creates a packet for sending or receiving data.

Example: Server

```
DatagramSocket socket = new Datagramsocket (5000);
byte [] buffer = new byte [1024];
Datagram Packet (buffer, buffer.length);
socket.recive (packet); // receives data.
Syste.out.println (new string (packet.getData(), 0,
  packet.getlength ()));
socket.close();
```

client:

client:

```
DatagramSocket socket = new DatagramSocket ();
byte [] buffer = "Hello, server!".getBytes ();
InetAddress.getByName ("localhost");
DatagramPacket packet = new DatagramPacket (buffer,
  buffer.length, address, 5000);
socket.send (packet); // send data
Socket.close ();
```

2. Diff bet^{w} Servlet and JSP

	JSP	Servlet
1.	JSP is a webpage scripting language generally used to create dynamic web content	Servlets are Java programs that are already compiled and which also create dynamic conte
2.	JSP is a tag based approach	Servlet is a pure Java code
3.	We can run JS at client side	cannot run Javascript at client
4.	Focuses more on displaying information.	Mainly focuses on information processing.
5.	JSP run slower than Servlet	Servlet runs faster than JSP
6.	JSP is Java in HTML	Servlet is html in Java.

3. Write GUI program to find sum of 2 no using swing components. Create Textfield for inputting 2 no's and a button named SUM. Program should produce sum on clicking SUM button.

Ans^{o}

```
import javax.swing.*;
public class SumCalculator
{
    public static void main (String[] args)
    {
        JFrame frame = new JFrame("Sum Calculator");
        frame.setSize(300, 150);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setLayout(new BoxLayout(frame.getContentPane(),
BoxLayout.Y_AXIS));

JTextField num1field = new JTextfield();
JTextfield num2field = new JTextfield();
JButton sumButton = new JButton("SUM");
JLabel resultlabel = new JLabel("Result: ");

sumButton.addActionListner(e ->
{
    try {
        double num1 = Double.parseDouble(nem2field.getText());
        double num2 = Double.parseDouble(num2field.getText());
        resultlabel.setText("Result: " + (num1 + num2));
~~resultlabel.setText("Result: " + (num1 + num2));~~
    } catch (NumberformatException ex) {
        JOptionPane.showMessageDialog(Frame, "Enter
valid numbers!", "Error", JoptionPane.ERROR_MESSAGE);
    }
});
frame.add(new JLabel("First number:"));
frame.add(num1field);
frame.add(new JLabel("Second Number:"));
frame.add(num2field);
frame.add(sumButton);
frame.add(resultLabel);
frame.setVisible(true);
}
}
```

OR,

```
import javax.swing.*;
public class SumCalculator {
    public static void main (String[] args) {
        JFrame frame = new JFrame ("Sum Calculator");
        JTextField num1 = new JTextField(), num2 = new JTextFi
        JLabel result = new JLabel ("Result : ");
        JButton sum = new JButton ("SUM");

        sum.AddActionListner (e -> {
            try {
                result.setText ("Result: " + (Double.parse
(num1.getText()) + Double.parseDouble (num2.getText()))
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog (frame, "Ent
numbers! ", "Error", JOptionPane.ERROR_MESSAGE);
            }
        });

        frame.setLayout (new BoxLayout (frame.getContentPa
        frame.add (new JLabel ("First Number:")); frame.ad
        frame.add (new JLabel ("Second Number:")); frame.
        frame.add (sum); frame.add (result);
        frame.setSize (300, 150);
        frame.setDefaultCloseOperation (JFrame.EXIT_ON_C
        frame.setVisible (true);
    }
}
```

4. What are the steps for creating client side and server side of applicn using TCP.

Ans° Steps for writing Client program

1. Open a socket

Socket client = new Socket (server_address, port_id);

This statement creates a socket for client computer. In IP address server program exectues and port numbe. in which server listens to the client.

2. Open an input stream and output stream to Socket

Scanner ins = new Scanner (client.getInputStream());
PrintWriter outs = new PrintWriter (client.getOutputStream(), true);

The first statement gets input stream of client's socket and opens Scanner on it. Similarly, the second statement gets output stream of client's socket and opens PrintWriter on it.

3. Read from and write to the sockets stream

rs = ins.nextLine();
outs.println(s);

The first statement reads a line from client's socket and puts it in variable 'rs' and the 2nd statm write the value of string variable 'ss' into client's socket

4. Close the streams

ins.close();
outs.close();

Once I/O is completed, above statm are used to close I/O stream of client's socket

5. Close the Socket

client.close()

This statement close the conn^t bet^w client & server computer.

⇒ Steps for writing Server program

1. Crede the object of ServerSocket class

ServerSocket server = new ServerSocket (2345);

This statement create socket for the server & the argument represents the port no to which the server listens.

2. Accept the connection from the client

Socket client = server.accept();

After creating the socket it enters into infinite loop and waits for the client connection by using accept method.

3. Get input and output streams of the Socket

Scanner ins = new Scanner (server.getInputStream());

PrintWriter outs = new PrintWriter(server.getOutputStream(), true);

1st stat^m gets input stream of server's socket and opens Scanner on it.
2nd gets output stream of server's socket and opens PrintWriter on it.

4. Read/Write Data to the Socket

rs = ins.nextline(), outs.println(ss);

1st reads a line from server's socket. 2nd write the value of string variable 'ss' into server's socket.

5. Close ~~steam~~ streams

ins.close(), outs.close()

6. Close Socket

Server.close()

client.close(),

It closes connection bet^w client & server computers.